

1. Diseño general y arquitectura de la calculadora

El objetivo principal es diseñar una calculadora que opere con números de 4 bits, manejando valores negativos mediante complemento a dos. El diseño se basa en una arquitectura de ruta de datos controlada por señales de entrada, sin utilizar operadores aritméticos de alto nivel.

1.1. Interfaz del Sistema

El sistema interactúa con el exterior mediante los siguientes puertos:

- **Entradas de datos:** op_1 (4 bits) como primer operando y op_2 (4 bits) como operando externo secundario.
- **Señales de control:**
 - Selector de operación (3 bits) que define la función a ejecutar (Reinicio, Suma, Resta, etc.).
 - Selector del segundo operando (1 bit) para alternar entre el operando externo op_2 y el resultado almacenado de la operación anterior.
 - Señal de confirmación/ejecución (1 bit) que actúa como habilitador para almacenar el resultado.
- **Salida:** Resultado (4 bits). Además, el bit de acarreo superior de la suma (overflow) se expone a través de un puerto de salida independiente para ser visualizado en un LED de la FPGA.

1.2. Diagrama de Bloques

La arquitectura se divide en tres subsistemas combinacionales y uno secuencial:

1. **Multiplexor de Entrada (2:1):** Selecciona el operando B entre op_2 y la retroalimentación del registro de salida.
2. **Ruta de Datos (ALU):** Conjunto de bloques combinacionales independientes (sumadores, desplazadores, lógica de ceros) que calculan simultáneamente todas las operaciones.
3. **Multiplexor de Salida (8:1):** Bloque que, mediante el selector de operación (3 bits), enruta la salida del bloque combinacional correspondiente hacia el registro final.
4. **Registro de Resultado:** Memoria de 4 bits que almacena la salida de la ALU en el flanco activo de la señal de ejecución.

2. Tablas de verdad, mapas de Karnaugh y expresiones booleanas

Para cumplir con la restricción de diseño, las operaciones se implementan a nivel de compuertas lógicas (AND, OR, NOT, XOR).

2.1. Suma (3'b001)

La suma se realiza mediante cuatro módulos de Sumadores Completos (Full Adders) conectados en cascada (Ripple Carry Adder).

Tabla de Verdad (Sumador Completo de 1 bit)

A_i	B_i	C_{in}	C_{out}	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Cuadro 1: Tabla de verdad para un sumador completo.

Mapas de Karnaugh y Ecuaciones

Para el bit de suma (S_i):

		$B_i C_{in}$			
		00	01	11	10
A_i	0	0	1	0	1
	1	1	0	1	0

$$S_i = A_i \oplus B_i \oplus C_{in}$$

Para el acarreo de salida (C_{out}):

		$B_i C_{in}$			
		00	01	11	10
A_i	0	0	0	1	0
	1	0	1	1	1

$$C_{out} = (A_i \cdot B_i) + (C_{in} \cdot (A_i \oplus B_i))$$

2.2. Resta (3'b010) y Resta Inversa (3'b011)

Ambas operaciones reutilizan el bloque sumador descrito en la sección anterior, aplicando el complemento a dos.

- **Resta ($A - B$):** Se invierten los bits de B ($\overline{B}$) y se fuerza un acarreo inicial $C_{in0} = 1$. La expresión es: $Result = A + \overline{B} + 1$.
- **Resta Inversa ($B - A$):** Se invierten los bits de A ($\overline{A}$) y se fuerza un acarreo inicial $C_{in0} = 1$. La expresión es: $Result = B + \overline{A} + 1$.

2.3. Shift Left (3'b100) y Shift Right (3'b101)

Los desplazamientos se realizan controlados por los dos bits menos significativos del operando B (B_1 y B_0).

Expresiones para Shift Left (Relleno con ceros a la derecha)

$$\begin{aligned}R_0 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_0) \\R_1 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_1) + (\overline{B_1} \cdot B_0 \cdot A_0) \\R_2 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_2) + (\overline{B_1} \cdot B_0 \cdot A_1) + (B_1 \cdot \overline{B_0} \cdot A_0) \\R_3 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_3) + (\overline{B_1} \cdot B_0 \cdot A_2) + (B_1 \cdot \overline{B_0} \cdot A_1) + (B_1 \cdot B_0 \cdot A_0)\end{aligned}$$

Expresiones para Shift Right (Relleno con ceros a la izquierda)

$$\begin{aligned}R_3 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_3) \\R_2 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_2) + (\overline{B_1} \cdot B_0 \cdot A_3) \\R_1 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_1) + (\overline{B_1} \cdot B_0 \cdot A_2) + (B_1 \cdot \overline{B_0} \cdot A_3) \\R_0 &= (\overline{B_1} \cdot \overline{B_0} \cdot A_0) + (\overline{B_1} \cdot B_0 \cdot A_1) + (B_1 \cdot \overline{B_0} \cdot A_2) + (B_1 \cdot B_0 \cdot A_3)\end{aligned}$$

2.4. Reinicio (3'b000)

La operación de reinicio se logra anulando forzosamente la salida del bloque, equivalente a multiplicar lógicamente cada bit por cero.

$$R_i = 0 \quad \text{para } i \in \{0, 1, 2, 3\}$$